

create.cliente

Projeto para criar aplicações clientes dentro de um formulário LCL do servidor.

- [Projeto para criar aplicações clientes dentro de um formulário LCL do servidor.](#)
 - [Introdução](#)
 - **Objetivo.**
 - [Diagrama do projeto.](#)
 - [Unit DataModules](#)

Introdução

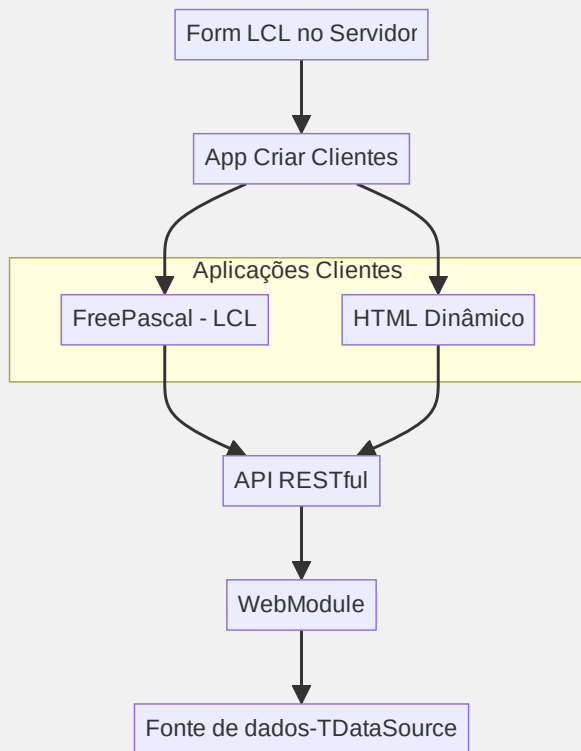
- O projeto \lar\carai visa desenvolver um gerador de aplicações clientes que interagem com serviços de backend, utilizando um framework flexível e modular. Com a crescente demanda por soluções que integrem diferentes tecnologias e plataformas, este projeto se propõe a facilitar a criação de aplicações clientes em diversas estruturas de framework, como Free Pascal - LCL, HTML dinâmico, Vue.js, AngularJS e ReactJS.
- A principal motivação por trás deste projeto é a necessidade de simplificar o processo de desenvolvimento de aplicações que se conectam a serviços de backend, permitindo que desenvolvedores criem rapidamente interfaces de usuário que se comunicam de forma eficiente com APIs. O gerador de aplicações clientes oferece uma interface intuitiva para configuração e personalização, permitindo que os usuários definam parâmetros essenciais, como o protocolo de comunicação, endereço do servidor, porta, e o nome do recurso (WebModule) que será utilizado.
- Atualmente, o projeto já implementou suporte para Free Pascal - LCL e HTML dinâmico, enquanto as implementações para Vue.js, AngularJS e ReactJS estão previstas para futuras versões, dependendo da demanda dos usuários. A arquitetura do sistema foi projetada para ser extensível, permitindo que novos frameworks sejam adicionados conforme necessário.
- Além disso, o projeto inclui um módulo de gerenciamento de dados que facilita a criação e edição de formulários, bem como o mapeamento de campos de banco de dados. Com um foco na usabilidade e na eficiência, o sistema é capaz de gerar automaticamente a estrutura de diretórios e arquivos necessários para cada tipo de aplicação cliente, garantindo que os desenvolvedores possam se concentrar na lógica de negócios e na experiência do usuário.
- Em resumo, o projeto \lar\carai representa uma solução inovadora para o desenvolvimento de aplicações clientes, promovendo a integração entre diferentes tecnologias e simplificando o processo de criação de interfaces de usuário dinâmicas e responsivas.

Objetivo.

- Desenvolver aplicativos clientes *http* ou *https* para as seguintes estruturas de framework:
 - *FreePascal - LCL*
 - Implementado.
 - *JavaScript*.
 - Implementado parcialmente.
 - *HTML dinâmico*.
 - Implementado.
 - *Vuejs*.
 - Não implementado.
 - *Angularjs*.
 - Não implementado.
 - *Reactjs*.
 - Não implementado.
- Notas:
 - Os frameworks para web browser não implementados na versão de lançamentos serão implementados caso exista demanda por parte dos usuários, visto que as aplicações cliente LCL e *HTML dinâmico* atendam as necessidades até o momento.
 - Internamente, o projeto já calcula as pastas nas quais as futuras implementações clientes serão criadas.

Diagrama do projeto.

Esse diagrama representa a arquitetura do sistema, destacando a interação entre o usuário, a aplicação cliente, o servidor, o banco de dados e o módulo de gerenciamento de dados, além de incluir os componentes específicos da aplicação cliente (Free Pascal - LCL e HTML Dinâmico).



- **Descrição do Diagrama.**

- *Servidor-Form LCL*: Formulário do servidor que aciona a aplicação para criar clientes.
- *App Criar Clientes*: A aplicação que permite a criação de clientes.
 - *Aplicações Cliente*: Um subgrupo que inclui:
 - *FreePascal - LCL*: Módulo http RestFul usando formulários LCL na aplicação cliente.
 - *HTML Dinâmico*: Módulo http/https RestFul escrito em HTML e JavaScript para criar formulários web na aplicação cliente.
- *API RESTful*: A interface que conecta as aplicações cliente ao servidor.
- *WebModule*: O módulo que gerencia as requisições da API.
- *Fonte de dados (TDataSource)*: Onde os dados dos clientes são armazenados.
 - **Notas.**
 - Cada webModule do servidor que não seja informada uma fonte de dados em particular, o sistema cria uma fonte de dados no formato json e usa o nome da tabela com extensão json.

Unit DataModules

- **Objetivo.**

- Unit usada para criar as regras na aplicação e definir o layout do formulário de entrada, bem como mapear o banco de dados usado pelo formulário.
 - **Nota:**
 - A tabela usada neste projeto é um arquivo json com o nome da aplicação cliente, no qual armazena todos os

- **Classe TCreateClientes**

- **Objetivo.**

- Contêm os parâmetros necessários para criar as aplicações clientes, bem como a definição da tela e do banco de dados usados para salvar os parâmetros para cada aplicação configurada.

- **Evento DmxScroller_Form1AddTemplate (Definição do formulário):**

- **Objetivo.**

- Formulário usado para edição dos recursos usados para criar os formulários, bem como mapeamento dos campos do banco de dados.

- Nota:06:28

- Os dados deste template serão armazenados no arquivo com nome do DataModule com a extensão (.json).

- **Descrição do formulário.**

- **Painel dos botões.**

- **Botões de ações.**

- Novo.

- *CmNewRecord* : Botão de ação coloca o registro no modo de edição para adicionar registro.

- Tags:

- *ChEA* : A sequência a seguir é o nome da ação.

- Exemplo de uso:

- *ChEA+CmNewRecord*+.

- Eventos:

- *DmxScroller_Form1.DoOnNewRecord*;

- Gravar.

- *CmUpdateRecord* : Botão de ação para adicionar o registro se o estado atual estiver no modo insert ou gravar as alterações se estiver no modo update.

- Tags:

- *ChEA* : A sequência a seguir é o nome da ação.

- Exemplo de uso:

- *ChEA+CmUpdateRecord*+.

- Eventos:

- *CmUpdateRecordExecute*.

- *DmxScroller_Form1.UpdateRec*;

- *DmxScroller_Form1.BeforeInsert*

- Neste momento, o sistema deve executar o módulo para criar a aplicação cliente.
- Nota:
 - Caso ocorra exceção, este evento deve retornar false.
- *DmxScroller_Form1BeforeUpdate*
 - Caso o registro seja alterado, a aplicação atual deve ser excluída e criada outra com os novos parâmetros.
 - Nota:
 - Caso ocorra exceção, este evento deve retornar false.
- Localizar.
 - *CmLocate* : Botão de ação localiza o registro para edição.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmLocate+*  .
 - Eventos:
 - *CmLocateExecute*.
 - *if Locate()= mrNo Then*
TMi_rtl.ShowMessage('Registro não localizado');
 - Notas:
 - Pesquisa na lista de todos os *WebModules* gerados por tipo de cliente.
- Excluir.
 - *CmDeleteRecord* : Botão de ação coloca o registro no modo de edição para adicionar registro.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmDeleteRecord+*  .
 - Eventos:
 - *CmDeleteRecordExecute*
 - *DmxScroller_Form1.DeleteRec;*
 - *DmxScroller_Form1BeforeInsert*
 - Neste momento, o sistema deve deletar o módulo cliente gerado antes de excluir o registro.
 - Notas:

- Caso ocorra exceção, este evento deve retornar false.
- Cancelar.
 - *Cancelar* : Botão de ação coloca o registro no modo de edição para adicionar registro.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+Cancelar+X*.
 - Eventos:
 - *CmCancelExecute*
 - *DmxScroller_Form1.Cancel;*
- Botões de navegação.
 - Primeiro registro.
 - *CmGoBofExecute* : Botão de ação para posicionar o dataset no primeiro registro.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmGoBofExecute+←* .
 - Eventos:
 - *DmxScroller_Form1.FirstRec;;*
 - Próximo registro.
 - *CmNextRecord* : Botão de ação coloca o registro no modo de edição para adicionar registro.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmNextRecord+→* .
 - Eventos:
 - *CmNextRecordExecute*.
 - *DmxScroller_Form1.NextRec;;*
 - Registro anterior.
 - *CmPrevRecord* : Botão de ação para posicionar no registro anterior.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmPrevRecord+←* .
 - Eventos:
 - *CmPrevRecordExecute*.
 - *DmxScroller_Form1.PrevRec;;*
 - Último Registro.

- *CmGoEof* : Botão de ação coloca o registro no modo de edição para adicionar registro.
- Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmGoEof~+* .
- Eventos:
 - *CmGoEofExecute*
 - *DmxScroller_Form1.LastRec;*
- Atualizar.
 - *CmRefresh* : Botão de ação para cancelar e ler o registro atual do arquivo.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação.
 - Exemplo de uso:
 - *ChEA+CmRefresh+* .
 - Eventos:
 - *CmRefreshExecute*
 - *DmxScroller_Form1.Refresh;*
- **Dados de acesso ao serviço:**
 - Protocolo do serviço no servidor:
 - *protocol*: Nome do campo.
 - Valores Possíveis
 - http.
 - https.
 - Eventos.
 - *OnCalcFields.*
 - Calcula a URL do serviço.
 - Endereço IP ou Nome do Host:
 - *Host* : Nome do campo.
 - Número da porta do serviço no servidor.
 - *Port* : Nome do campo.
 - Nome do recurso (*TMi_rtl_web_module*) do servidor:
 - *TMi_rtl_web_module* : Nome do campo.
 - Eventos.
 - *OnCalcFields.*
 - Calcula a URL do serviço.
 - Notas:
 - O nome do *webModule* deve ser passado como parâmetro ao executar o formulário *TCreateclientesForm*.
 - Nome da URL do serviço
 - *URL* : Nome do campo.

- Tag
 - CharDefaultExpression+'protocol'+:./+
'Host'+/+'Tmi_rtl_web_module'.
- **Parâmetros para criar aplicações clientes.**
 - *Número sequencial:*
 - *id* : Nome do campo;
 - Tags:
 - *ChFN*:
 - Indica que a próxima sequência é o nome do campo.
 - *CharAccSkip*:
 - Ao pressionar a tecla tab, o campo não é selecionado.
 - *CharAccReadOnly*:
 - Indica que o campo é somente para leitura.
 - *CharPfInKeyPrimary*:
 - Indica que o campo é um campo de chave primária. É usado na cláusula WHERE de uma instrução de atualização.
 - *CharPfInKeyPrimaryAutoIncrement*:
 - Indica que o campo é um campo autoincremental. É usado em uma instrução de atualização.
 - *ChH*:
 - É usado para documentar o campo. Indica que todo o texto até o próximo caractere de controle será o conteúdo do campo HelpCtx_Hint.
 - *Nome da pasta root onde se criarão as aplicações clientes:*
 - *PathRoot*: Nome do Campo.
 - *CmLocatePathRoot* : Botão de ação para selecionar pasta.
 - Tags:
 - *ChEA* : A sequência a seguir é o nome da ação. A mesma deve ser implementada no formulário LCL.
 - Exemplo de uso:
 - *ChEA+CmLocatePathRoot*+.
 - Eventos.
 - *OnEnterField*.
 - Seleciona a pasta root onde a aplicação será criada.
 - Obs.: O padrão da pasta root é a pasta do executável da aplicação servidora.
 - *Tipo de aplicação.*

- *typApp* : Nome do campo.
 - Valores possíveis.
 1. App LCL : sim.
 2. App Javascript: não.
 3. App Dynamic_html:sim.
 4. App VueJS: não.
 5. App AngularJS: não.
 6. App ReactJS: não.
 - Eventos:
 - *OnCalcFields*
 - Calcular o nome da pasta destino da aplicação.
- *Nome da subpasta da aplicação cliente usada para gerar a aplicação.*
 - *PathClient*: Nome do Campo
 - Tags :
 - *CharAccReadOnly*:
 - Indica que o campo é somente para leitura.
 - **Exemplo de árvore gerada:**

```

.
├── lcl
│   └── templates
│       ├── mi rtl.web.module.dfm
│       ├── mi rtl.web.module.pas
│       ├── mi rtl.web.module.form.dfm
│       └── mi rtl.web.module.form.pas
├── dynamic_html
│   ├── css
│   │   ├── color_tons_amarelo.css
│   │   ├── color_tons_dark.css
│   │   ├── color_tons_de_azul_ceu.css
│   │   ├── color_tons_de_cinza.css
│   │   ├── color_tons_de_dark_claro.css
│   │   ├── color_tons_de_lilas_claro.css
│   │   ├── color_tons_de_verde_claro.css
│   │   ├── MiEditForm.css
│   │   └── MiUiDmxScroller.css
│   ├── js
│   │   ├── MiMessageBox.js
│   │   ├── MiThemeDialog.js
│   │   ├── MiConsts.js
│   │   └── MiMethods.js
│   │       ├── MiUiDmxScroller.js
│   │       └──
│   ├── MiUiDmxScrollerForm.js
│   ├── MiUiDmxScrollerForm.MiAbstractDialogs.js
│   ├── MiUiDmxScrollerForm.MiDialogs.js
│   │   ├── MiDates.js
│   │   ├── MiMaskEdit.js
│   │   └── MiEditForm.js
│   ├── templates
│   │   └── MiEditForm.html
│   └── Tmi_rtl_web_module.form.html

```

- Nome do arquivo destino do código gerado.
 - *fileNameResult* : Nome do campo.
 - Tags :
 - *CharAccReadOnly*:
 - Indica que o campo é somente para leitura.
 - Notas:
 - O nome do arquivo destino será o mesmo nome do WebModule server com a extensão compatível com a aplicação destino.
 - Exemplo:
 - Módulo servidor:
 - Classe **Tmi_rtl_web_module**

representa um serviço do servidor. -
O arquivo cliente a ser gerado para o módulo servidor **Tmi_rtl_web_module** é:
- **Tmi_rtl_web_module.pas** se app FreePascal - LCL. -
Tmi_rtl_web_module.js se app javascript. - **Tmi_rtl_web_module.html** se app HTML dinâmico.
- **Tmi_rtl_web_module.vue** se App VueJS.
- **Tmi_rtl_web_module** App AngularJS. -
Tmi_rtl_web_module.module.ts: Configuração do módulo principal da aplicação. -
Tmi_rtl_web_module.component.ts: Componente raiz da aplicação. -
Tmi_rtl_web_module.component.html: Template do componente raiz. -
Tmi_rtl_web_module.component.js: O transpilador Ângula gera o javascript a partir do Typescript -
Tmi_rtl_web_module.service.ts: Serviço para fazer chamadas HTTP para a API RESTful. -
Tmi_rtl_web_module.jsx App ReactJS.

- A versão de lançamento do projeto maricarai vai gerar aplicações clientes somente para LCL e HTML dinâmico, visto que as mesmas atendem todas as necessidades, porém preparei o projeto para que no futuro alguém interessado queira implementar destinos para os frameworks acima.

▪ Desenho do formulário

- Evento: **DmxScroller_Form1AddTemplate**.

```

~+ &Novo      ~^TCmNewRecord~✓ &Gravar      ~^TCmUpdate
&Cancelar ~^TcmCancel
~← P&primeiro ~^TCmGoBof~→ Pró&ximo ~^TCmNextRecord
A&tualizar~^TCmRefresh

'~~'
'~  DADOS DE ACESSO AO SERVIÇO~'
'~  Protocolo:~\ssssssssss'+ChFN+'protocol'+ChDfC+'ht

'~      Host:~\ssssssssssssssssssssssssssssss'+ChFN+'
'~  Porta:~\#####'+ChFN+'port'+chH+'Número da porta
~\ssssssssssssssssssssssssssssss'+chFN+'Tmi_rtl_web_mod
servidor.', comentei o help pq a linha não pode ser mai

'~      URL:
~\ssssssssssssssssssssssssssssssssssssssssssssssssssss
ChFN+'url'+ChDfE+'protocol + ':'//'+ host + ':'
cliente para comunicar-se com o servidor.'

'~      PARÂMETROS PARA CRIAR APLICAÇÕES CLIENTES~'
'~      ID:
~\LLLLLL'+ChFN+'id'+ChAS+ChARO+CharPfInKeyPrimary+CharP
'~      Selecione a aplicação destino: ~\'+CreateEnu
NewSI
NewSI
NewSI
NewSI
NewSI
NewSI
nil))
ChFN+
'typA
ChH+
'Tipo

'~      Nome da pasta raiz dos
clientes:~\ssssssssssssssssssssssssssssssssssssssssss
ChFN+'pathRoot'+ChDfC+'./clientes'+ChH+'Usado com
cliente.'+commandIcon('pathRoot','cmSelectDirectoryDial

'~      Nome do arquivo destino: ~\ssssssssss
ChFN+'fileNameResult'+ChARO+ChDfE+'Tmi_rtl_web_mo
aplicação cliente. '
'~~'
command(' ~Gerar formulário~ ', 'cmBuildCustomer
'~~'

```

- **TCreateClientes.BuildCustomerFormFromTemplate(aEnClientsApplication:TEnClientsApplication);**

- *Objetivo:*

- Método usado para criar aplicações clientes do formulário passado pelo método TCreateClientes.SetFormServer.

- *Parâmetros passados para o gerador de aplicação cliente:*

- Formulário com as coordenadas dos controles.
 - `_FormServer: TForm;`
 - `_Mi_lcl_ui_ds_Form: TMi_lcl_ui_ds_Form;`
 - `_Mi_rtl_web_module: TMi_rtl_WebModule_base;`
 - *Obs.:*
 - Necessários para criar HTML dinâmico, visto que o mesmo usa valor absoluto dos controles da visão no caso de aplicação do tipo `en_app_dynamic_html`.
 - *PathRoot:* Nome do Campo.
 - Usado para identificar a pasta html que o servidor está servindo.
 - *typApp* : Nome do campo.
 - Usado para selecionar o tipo de aplicação.
 - *PathClient:* Nome do Campo
 - Pasta da aplicação cliente.
 - *fileNameResult* : Nome do campo.
 - Nome do arquivo a ser gerado, onde a extensão é obtida pelo parâmetro *typApp*.
- *Procedimentos que executam a classe que gera os formulários clientes.*
 - *Procedure Create_app_cliente_lcl();*
 -
 - *Procedure Create_app_dynamic_html();*
 - Classe usada para criar a aplicação:
 - `TMi_web_js_Form`.

